

Activity 9 - Price mixed order lines with traits and iterators

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K3/A3

Specification

trait Priced { unit_cents, quantity, line_cents (default, checked) }. order_total<T: Priced>(&[T]) and mixed_total(&[&dyn Priced]) return Option<u64> (None on overflow; empty is Some(0)). names_at_least(&[Product], min_cents) -> Vec<&str> in input order.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain traits, default methods, generics, trait objects (dyn), closures and iterator adaptors to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is samples/01_trait_basics.rs from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on traits, default methods, generics, trait objects (dyn), closures and iterator adaptors with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles traits, generics and iterators with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: trait Priced { unit_cents, quantity, line_cents (default, checked) }. order_total<T: Priced>(&[T]) and mixed_total(&[&dyn Priced]) return Option<u64> (None on overflow; empty is Some(0)). names_at_least(&[Product], min_cents) -> Vec<&str> in input order. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Price mixed order lines with traits and iterators" in starter/src so that the supplied tests pass. Implement with iterator adaptors (iter, filter, map, try_fold); no index-based for loops. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_trait_basics.rs

Run: rustc --edition 2021 01_trait_basics.rs && ./01_trait_basics

// Sample: one trait, two types, generic and dynamic dispatch.

```
trait Priced {
    fn unit_cents(&self) -> u64;
    fn quantity(&self) -> u32;
    fn line_cents(&self) -> u64 {
        self.unit_cents() * u64::from(self.quantity())
    }
}

struct Product {
    unit_cents: u64,
    qty: u32,
}

struct Service {
    rate_cents: u64,
    hours: u32,
}

impl Priced for Product {
    fn unit_cents(&self) -> u64 {
        self.unit_cents
    }
    fn quantity(&self) -> u32 {
        self.qty
    }
}

impl Priced for Service {
    fn unit_cents(&self) -> u64 {
        self.rate_cents
    }
    fn quantity(&self) -> u32 {
        self.hours
    }
}

fn show<T: Priced>(label: &str, line: &T) {
    println!("{label}: {} cents", line.line_cents());
}

fn main() {
    let cable = Product {
        unit_cents: 899,
        qty: 2,
    };
    let setup = Service {
        rate_cents: 5000,
        hours: 2,
    };
    show("cable", &cable);
    show("setup", &setup);
    let order: Vec<&dyn Priced> = vec! [&cable, &setup];
```

```

    let total: u64 = order.iter().map(|line| line.line_cents()).sum();
    println!("mixed order total: {total} cents");
}

```

Expected output:

```

cable: 1798 cents
setup: 10000 cents
mixed order total: 11798 cents

```

02_closures_iterators.rs

Run: `rustc --edition 2021 02_closures_iterators.rs && ./02_closures_iterators`

// Sample: closures and iterator chains instead of index loops.

```

fn main() {
    let prices = [899u64, 2499, 1299, 350];
    let threshold = 1000;
    let expensive: Vec<u64> =
        prices.iter().copied().filter(|p| *p >= threshold).collect();
    println!("at least {threshold}: {expensive:?}");
    let doubled: Vec<u64> = prices.iter().map(|p| p * 2).collect();
    println!("doubled: {doubled:?}");
    let total = prices.iter().try_fold(0u64, |acc, p| acc.checked_add(*p));
    println!("checked total: {total:?}");
    let add_gst = |cents: u64| cents * 109 / 100;
    println!("899 with 9% GST: {}", add_gst(899));
}

```

Expected output:

```

at least 1000: [2499, 1299]
doubled: [1798, 4998, 2598, 700]
checked total: Some(5047)
899 with 9% GST: 979

```

Learn more

Rust Book: Traits: Defining Shared Behavior: <https://doc.rust-lang.org/book/ch10-02-traits.html>

Rust Book: Closures: <https://doc.rust-lang.org/book/ch13-01-closures.html>

Rust Book: Processing a Series of Items with Iterators:

<https://doc.rust-lang.org/book/ch13-02-iterators.html>

Programiz: Rust Trait: <https://www.programiz.com/rust/trait>

Programiz: Rust Generics: <https://www.programiz.com/rust/generics>

Programiz: Rust Closure: <https://www.programiz.com/rust/closure>

Programiz: Rust Iterators: <https://www.programiz.com/rust/iterators>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php